Python activity based learning

CASE STUDIES

1.Fuel Consumption and Route Comparison

2.Progressive Water Tariff Analysis

3.Advanced Numeric Pattern Generator

4.Password Policy Validation Engine

5.Smart Parking Fee Management

6. Multi-Day Water Tank Simulation

7.Student Performance Trend Analysis

8. Traffic Violation Penalty Calacutatior

9. University Scholarship Allocation

10.University Admission Decision Engine

1.Fuel Consumption and Route Comparison

Problem Statement: *How can we compare different routes based on distance, fuel consumption, and fuel cost to find the most economical route?*

Algorithm: Fuel Consumption and Route Comparison

1. ***Start***
2. *Enter the number of routes to compare.*
3. *Enter the distance of each route in kilometers.*
4. *Enter the vehicle's mileage (km/litre).*
5. *Enter the fuel price per litre.*
6. *Calculate fuel required for each route:*
   Fuel Required = Distance / Mileage
7. *Calculate fuel cost for each route:*
   Fuel Cost = Fuel Required × Fuel Price
8. *Compare the fuel costs of all routes.*
9. *Identify the route with the lowest fuel cost.*
10. *Display the distance, fuel consumption, fuel cost, and best route.*

11. ***Stop***

Code:

```
mileage = float(input("Enter vehicle mileage (km/litre): "))
fuel_price = float(input("Enter fuel price per litre (Rs): "))
n = int(input("Enter number of routes: "))

lowest_cost = float('inf')
best_route = ""

for i in range(1, n + 1):
    distance = float(input(f"Enter distance of Route {i} (km): "))

    fuel = distance / mileage
    cost = fuel * fuel_price

    print("\nRoute", i)
    print("Distance:", distance, "km")
    print("Fuel Required:", round(fuel, 2), "litres")
    print("Fuel Cost: Rs.", round(cost, 2))

    if cost < lowest_cost:
        lowest_cost = cost
        best_route = "Route " + str(i)

print("\n--- Best Route ---")
print("Most economical route:", best_route)
print("Lowest fuel cost: Rs.", round(lowest_cost, 2))
```

Expected Output:

Enter vehicle mileage (km/litre): 100

Enter fuel price per litre (Rs): 25

Enter number of routes: 1

Enter distance of Route 1 (km): 3

Route 1

Distance: 3.0 km

Fuel Required: 0.03 litres

Fuel Cost: Rs. 0.75

--- Best Route ---

Most economical route: Route 1

Lowest fuel cost: Rs. 0.75

Actual Output:

Enter vehicle mileage (km/litre): 100

Enter fuel price per litre (Rs): 25

Enter number of routes: 1

Enter distance of Route 1 (km): 3

Route 1

Distance: 3.0 km

Fuel Required: 0.03 litres

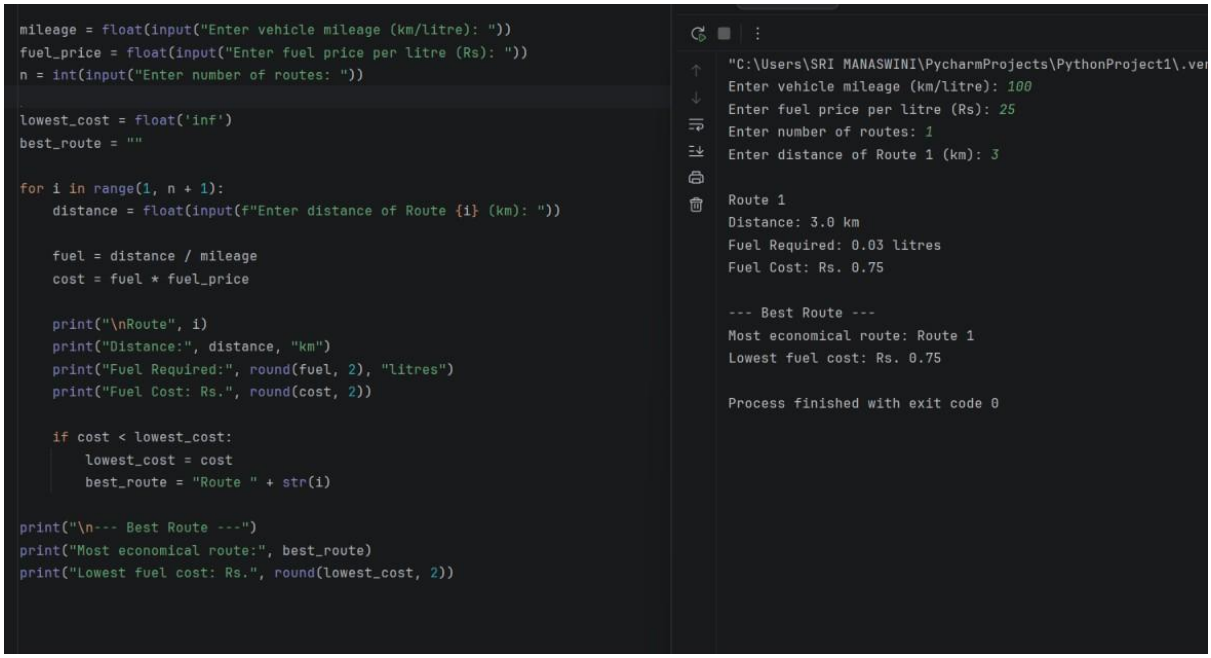
Fuel Cost: Rs. 0.75

--- Best Route ---

Most economical route: Route 1

Lowest fuel cost: Rs. 0.75

Output Screenshot:



```
mileage = float(input("Enter vehicle mileage (km/litre): "))
fuel_price = float(input("Enter fuel price per litre (Rs): "))
n = int(input("Enter number of routes: "))

lowest_cost = float('inf')
best_route = ""

for i in range(1, n + 1):
    distance = float(input(f"Enter distance of Route {i} (km): "))

    fuel = distance / mileage
    cost = fuel * fuel_price

    print("\nRoute", i)
    print("Distance:", distance, "km")
    print("Fuel Required:", round(fuel, 2), "litres")
    print("Fuel Cost: Rs.", round(cost, 2))

    if cost < lowest_cost:
        lowest_cost = cost
        best_route = "Route " + str(i)

print("\n--- Best Route ---")
print("Most economical route:", best_route)
print("Lowest fuel cost: Rs.", round(lowest_cost, 2))
```

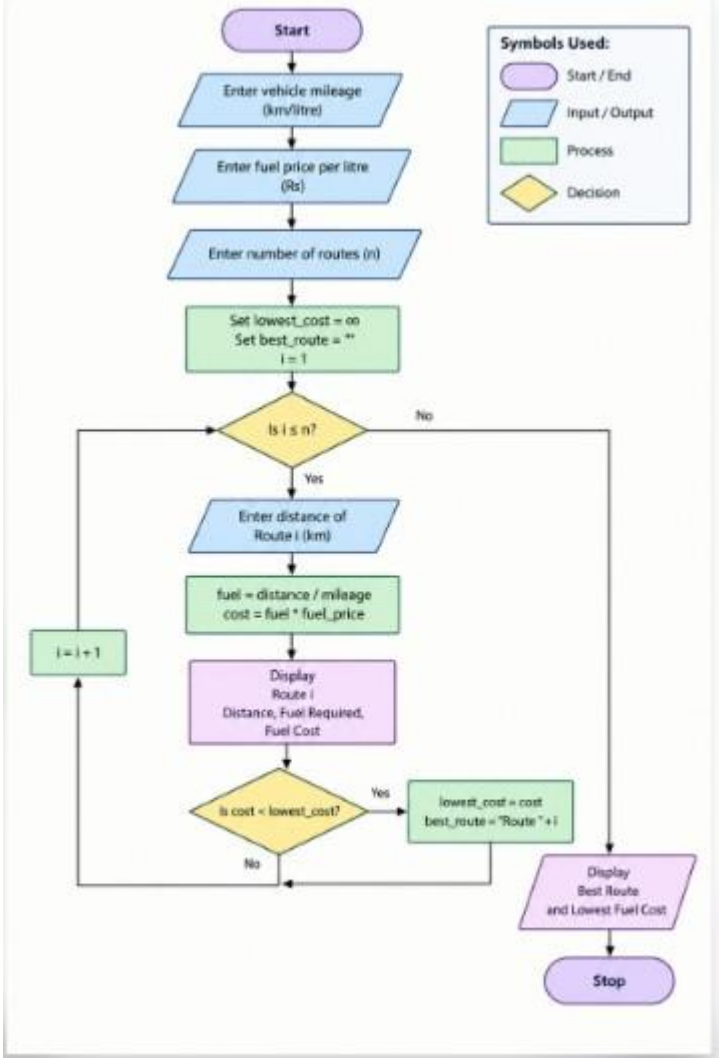
```
"C:\Users\SRI MANASWINI\PycharmProjects\PythonProject1\.ve
Enter vehicle mileage (km/litre): 100
Enter fuel price per litre (Rs): 25
Enter number of routes: 1
Enter distance of Route 1 (km): 3

Route 1
Distance: 3.0 km
Fuel Required: 0.03 litres
Fuel Cost: Rs. 0.75

--- Best Route ---
Most economical route: Route 1
Lowest fuel cost: Rs. 0.75

Process finished with exit code 0
```

Flowchart:



Objectives:

Calculate the fuel required for each route.

Estimate the fuel cost for each route using the given fuel price.

Compare the costs of multiple routes.

Display the route with the lowest fuel cost.

Practise Python input, loops, arithmetic operations, conditional statements, and formatted output.

Assumptions

Vehicle mileage remains constant for all routes.

Fuel price per litre is the same for all routes.

Fuel consumption depends only on route distance; traffic, road condition, hills, and idling are not considered.

The user enters positive mileage, positive fuel price, a positive number of routes, and non-negative route distances.

The most economical route means the route with the lowest calculated fuel cost, not necessarily the shortest travel time.

Test Cases:

Test	*Mileage (km/L)*	*Fuel price (₹/L)*	*Route distances (km)*	*Expected result*
1	*15*	*100*	*60, 90, 45*	*Route 3; lowest cost ₹300.00*
2	*10*	*100*	*20, 30*	*Route 1; lowest cost ₹200.00*
3	*20*	*95*	*100, 50, 75*	*Route 2; lowest cost ₹237.50*
4	*0*	*100*	*50*	*Validation message for invalid mileage*

Pseudocode:

```
BEGIN
    INPUT mileage
    INPUT fuel_price
    INPUT number_of_routes

    lowest_cost ← infinity
    best_route ← empty string

    FOR route_number FROM 1 TO number_of_routes
        INPUT distance
        fuel_required ← distance / mileage
        fuel_cost ← fuel_required * fuel_price

        DISPLAY distance, fuel_required, fuel_cost

        IF fuel_cost < lowest_cost THEN
```

```
        lowest_cost ← fuel_cost
        best_route ← "Route " + route_number
      END IF
   END FOR

   DISPLAY best_route
   DISPLAY lowest_cost
END
```

Conclusion:

The Fuel Consumption and Route Comparison program calculates the estimated fuel required and fuel cost for each route. By comparing these costs, it identifies the route with the lowest fuel expense. The project demonstrates the practical use of Python variables, input/output statements, loops, arithmetic calculations, and if statements. Actual fuel use may differ because traffic and road conditions are not included.

2.PROGRESSIVE WATER TARIFF ANALYSIS

Problem Statement:

Water utilities often charge customers according to how much water they use. A progressive water tariff system applies higher rates to higher levels of consumption. Develop a Python program that accepts a household's water usage in kilolitres (kL), calculates the bill using progressive slabs, and displays the usage, charge for each slab, total bill, and average cost per kilolitre.

Objectives:

Understand how progressive billing slabs work.

Accept and validate water consumption input.

Calculate charges slab by slab without charging the higher rate for all units.

Display a clear itemized bill and total amount.

Test the program with different consumption values.

Assumptions:

Water usage is entered in kilolitres (kL) and may be a decimal value.

The tariff rates below are illustrative for this academic case study, not official local rates.

First 10 kL: ₹5 per kL.

Next 10 kL (more than 10 and up to 20 kL): ₹8 per kL.

Usage above 20 kL: ₹12 per kL.

There is no fixed charge, tax, penalty, or minimum bill included.

Negative usage is invalid.

Algorithm:

Start.

Read water consumption in kL.

If the input is negative or not numeric, display an error and stop.

Set the first-slab quantity to min(usage, 10) and calculate its charge at ₹5/kL.

Set the second-slab quantity to min(max(usage − 10, 0), 10) and calculate its charge at ₹8/kL.

Set the third-slab quantity to max(usage − 20, 0) and calculate its charge at ₹12/kL.

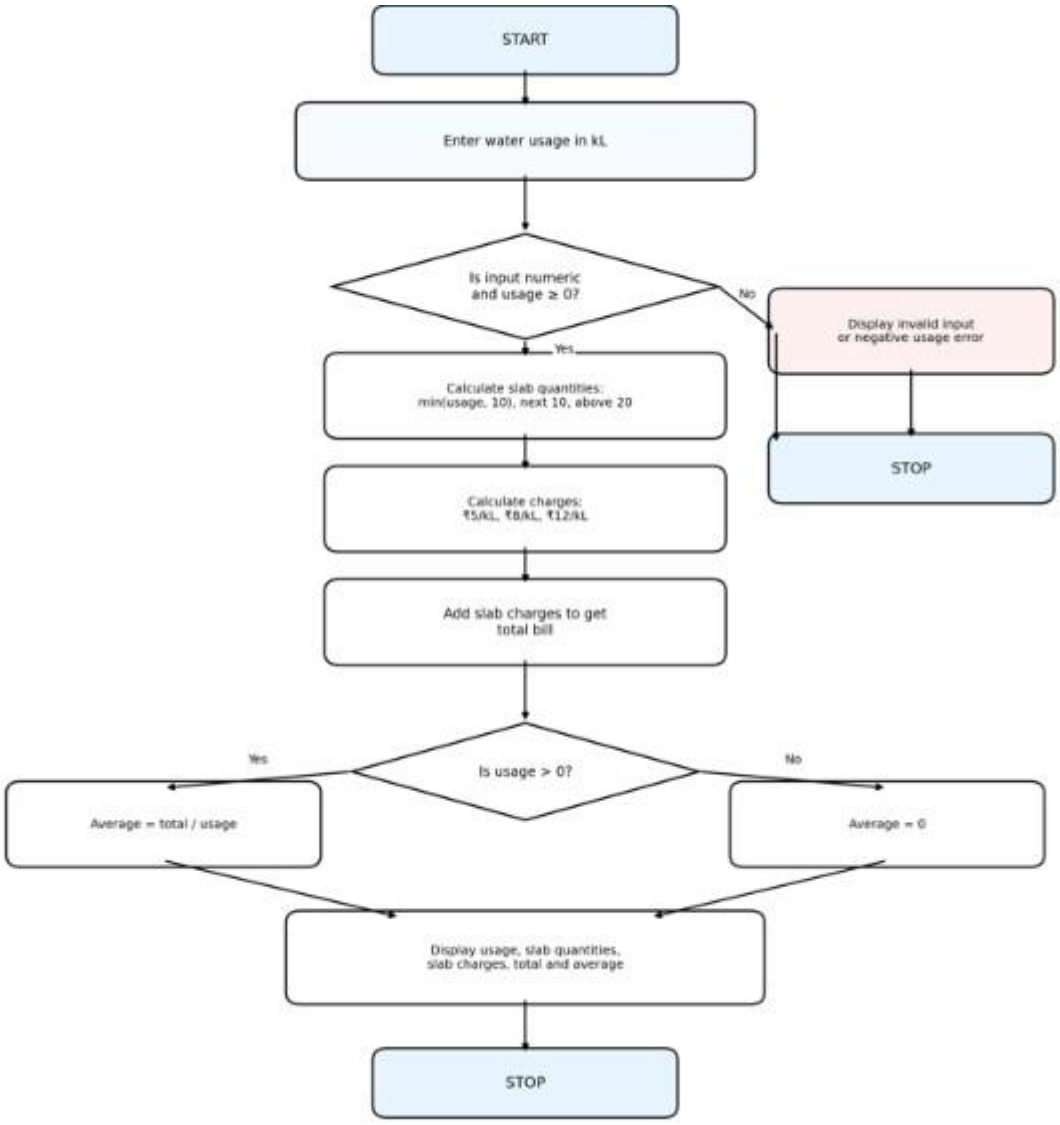
Add the three slab charges to obtain the total bill.

If usage is greater than zero, calculate average cost per kL; otherwise display ₹0.00.

Display usage, slab quantities, slab charges, total bill, and average cost.

Stop.

Flowchart:



Pseudocode:

BEGIN
INPUT usage
IF usage is not numeric OR usage < 0 THEN
PRINT "Invalid water usage"
STOP

```
END IF

slab1_units = MIN(usage, 10)
slab2_units = MIN(MAX(usage - 10, 0), 10)
slab3_units = MAX(usage - 20, 0)

charge1 = slab1_units * 5
charge2 = slab2_units * 8
charge3 = slab3_units * 12
total = charge1 + charge2 + charge3

IF usage > 0 THEN
average = total / usage
ELSE
average = 0
END IF

PRINT slab units, slab charges, total, average
END
```

Python Source Code:

```
def calculate_water_bill(usage):
if usage < 0:
raise ValueError("Water usage cannot be negative.")

slab1_units = min(usage, 10)
slab2_units = min(max(usage - 10, 0), 10)
slab3_units = max(usage - 20, 0)

rate1, rate2, rate3 = 5, 8, 12
charge1 = slab1_units * rate1
charge2 = slab2_units * rate2
charge3 = slab3_units * rate3
total = charge1 + charge2 + charge3
average = total / usage if usage > 0 else 0

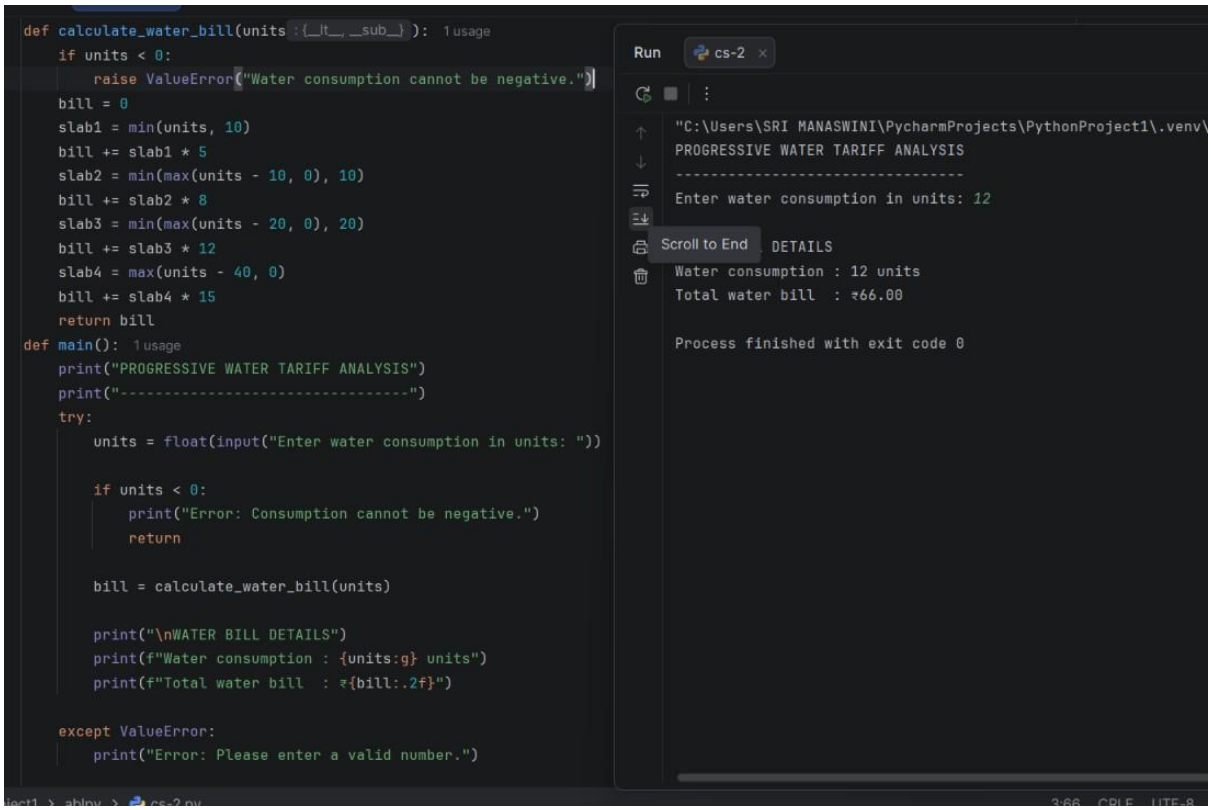
return {
"usage": usage,
"slab1_units": slab1_units, "charge1": charge1,
"slab2_units": slab2_units, "charge2": charge2,
"slab3_units": slab3_units, "charge3": charge3,
```

"total": total, "average": average
}

try:
usage = float(input("Enter water usage in kL: "))
bill = calculate_water_bill(usage)

print("\n----- WATER BILL -----")
print(f"Water usage: {bill['usage']:.2f} kL")
print(f"First 10 kL at Rs. 5/kL: {bill['slab1_units']:.2f} kL = Rs. {bill['charge1']:.2f}")
print(f"Next 10 kL at Rs. 8/kL: {bill['slab2_units']:.2f} kL = Rs. {bill['charge2']:.2f}")
print(f"Above 20 kL at Rs. 12/kL: {bill['slab3_units']:.2f} kL = Rs. {bill['charge3']:.2f}")
print(f"Total bill: Rs. {bill['total']:.2f}")
print(f"Average cost per kL: Rs. {bill['average']:.2f}")
except ValueError as error:
print("Invalid input:", error)

Output Screenshot:



Test Cases:

Input usage (kL)	Slab 1 charge	Slab 2 charge	Slab 3 charge	Expected total
0	₹0.00	₹0.00	₹0.00	₹0.00
8	₹40.00	₹0.00	₹0.00	₹40.00
15	₹50.00	₹40.00	₹0.00	₹90.00
25	₹50.00	₹80.00	₹60.00	₹190.00
-2	—	—	—	Error: negative usage

Sample Output:

Enter water usage in kL: 25

Water usage: 25.00 kL
First 10 kL at Rs. 5/kL: 10.00 kL = Rs. 50.00
Next 10 kL at Rs. 8/kL: 10.00 kL = Rs. 80.00
Above 20 kL at Rs. 12/kL: 5.00 kL = Rs. 60.00
Total bill: Rs. 190.00
Average cost per kL: Rs. 7.60

Output screenshot note: Run the Python program in IDLE, VS Code, or a terminal, enter 25, and take a screenshot of the displayed output for submission.

Conclusion:

The Progressive Water Tariff Analysis program calculates a household water bill using three consumption slabs. It demonstrates how a progressive tariff charges different rates for different portions of usage, validates negative input, and presents an itemized bill. The sample rates are for learning purposes and can be replaced with the tariff rates provided by a local water utility.

3.ADVANCED NUMERIC PATTERN GENERATOR

Problem Statement:

Develop a menu-driven Python application that generates several numeric patterns based on the user's selected pattern type and number of rows. The program should validate user input, display the selected pattern clearly, and allow the user to generate another pattern or exit.

Objectives:

Practice nested loops and number sequencing in Python.

Generate multiple numeric patterns from a single menu-driven program.

Use functions to keep the program modular and easy to maintain.

Validate menu choices and row counts to prevent invalid input.

Test the program with different pattern choices and row sizes.

Assumptions:

The program runs in Python 3.8 or later in a terminal or IDE.

The row count must be an integer from 1 to 20.

The user selects one of the five available patterns.

Patterns are printed using spaces between numbers; alignment may vary with larger numbers.

The program repeats until the user chooses Exit.

Algorithm:

1. Start the program.

2. Display the menu of numeric patterns and the Exit option.

3. Read the user's menu choice.

4. If the choice is Exit, display a closing message and stop.

5. If the choice is invalid, show an error message and return to the menu.

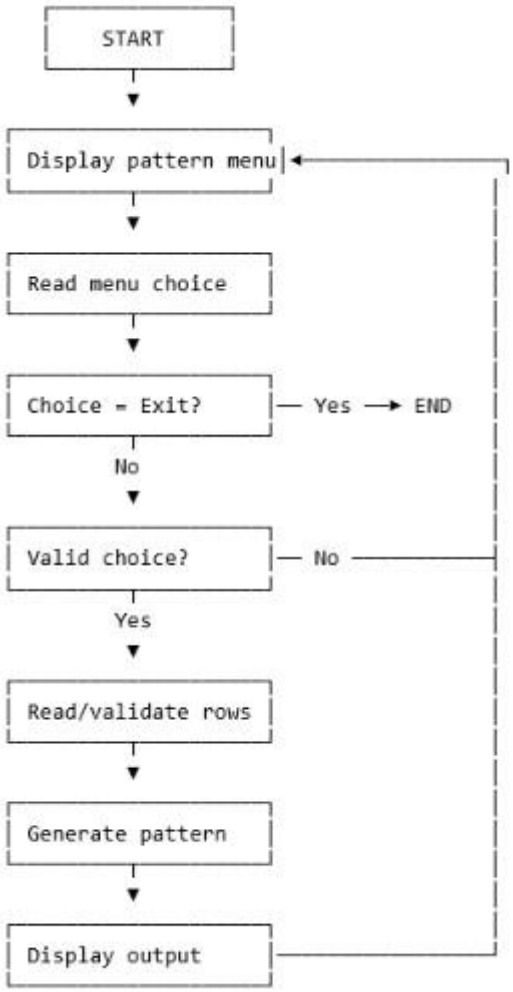
6. Read the number of rows and validate that it is an integer from 1 to 20.

7. Call the function for the selected pattern.

8. Print the generated pattern.

9. Return to the menu and repeat.

Flowchart:



Pseudocode:

```
BEGIN
  REPEAT
    DISPLAY menu
    INPUT choice

    IF choice = 6 THEN
      DISPLAY "Thank you"
      STOP
    END IF

    IF choice is not between 1 and 5 THEN
```

```
      DISPLAY "Invalid choice"
      CONTINUE
    END IF

    INPUT rows
    IF rows is not an integer from 1 to 20 THEN
      DISPLAY "Invalid row count"
      CONTINUE
    END IF

    CALL the function corresponding to choice
    DISPLAY generated pattern
  UNTIL user exits
END
```

Python Source Code:

```
def increasing_triangle(rows):
  for i in range(1, rows + 1):
    print(*range(1, i + 1))


def repeated_number_triangle(rows):
  for i in range(1, rows + 1):
    print(*([i] * i))


def floyd_triangle(rows):
  number = 1
  for i in range(1, rows + 1):
    values = []
    for _ in range(i):
      values.append(number)
      number += 1
    print(*values)
```

```
def palindrome_pyramid(rows):
    for i in range(1, rows + 1):
        left = list(range(1, i + 1))
        right = list(range(i - 1, 0, -1))
        print(" " * (rows - i), *(left + right))

def diamond_pattern(rows):
    for i in range(1, rows + 1):
        values = list(range(1, i + 1)) + list(range(i - 1, 0, -1))
        print(" " * (rows - i), *values)
    for i in range(rows - 1, 0, -1):
        values = list(range(1, i + 1)) + list(range(i - 1, 0, -1))
        print(" " * (rows - i), *values)

def main():
    patterns = {
        1: ("Increasing number triangle", increasing_triangle),
        2: ("Repeated-number triangle", repeated_number_triangle),
        3: ("Floyd's triangle", floyd_triangle),
        4: ("Palindrome number pyramid", palindrome_pyramid),
        5: ("Diamond number pattern", diamond_pattern),
    }

    while True:
        print("\nADVANCED NUMERIC PATTERN GENERATOR")
        for key, (name, _) in patterns.items():
            print(f"{key}. {name}")
        print("6. Exit")
```

```
        try:
            choice = int(input("Choose pattern (1-6): "))
        except ValueError:
            print("Please enter a whole number.")
            continue

        if choice == 6:
            print("Thank you for using the generator.")
            break
        if choice not in patterns:
            print("Invalid choice. Select a number from 1 to 6.")
            continue

        try:
            rows = int(input("Enter number of rows (1-20): "))
        except ValueError:
            print("Rows must be a whole number.")
            continue

        if not 1 <= rows <= 20:
            print("Invalid row count. Enter a value from 1 to 20.")
            continue

        print("\nOutput:")
        patterns[choice][1](rows)

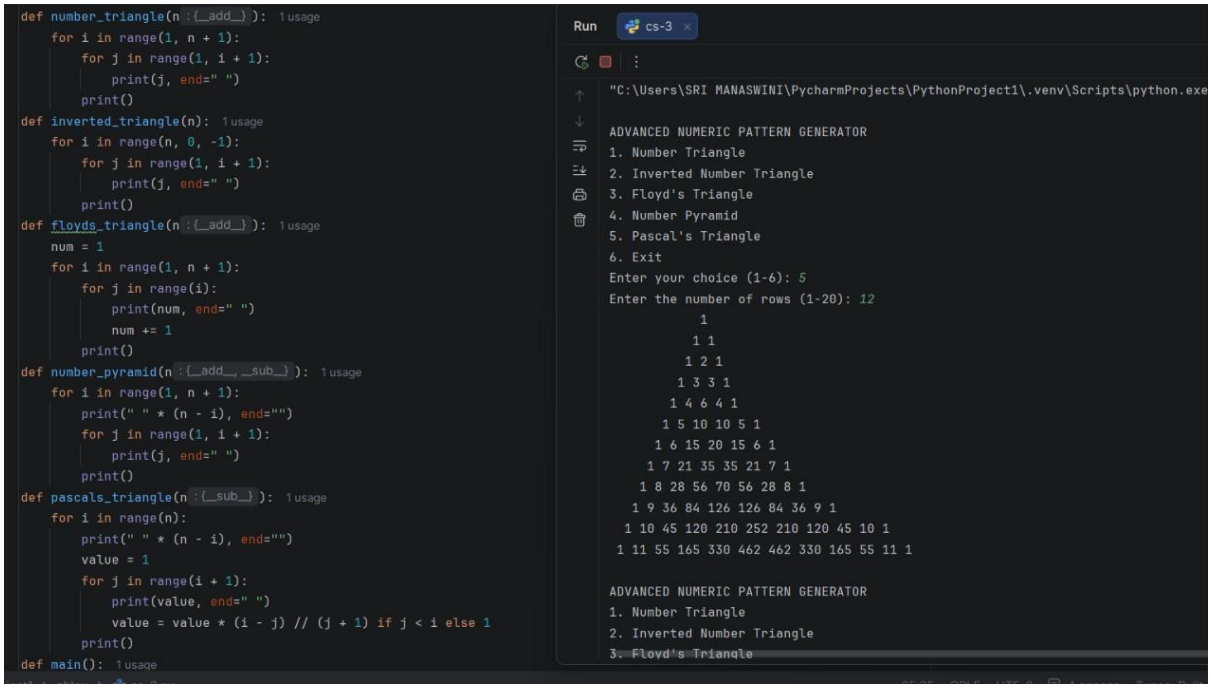
if __name__ == "__main__":
```

main()

Test Cases:

Test	Input	Expected Result	Status
1	Choice 1, rows 5	Increasing triangle with 5 rows	Pass
2	Choice 2, rows 3	Rows: 1; 2 2; 3 3 3	Pass
3	Choice 3, rows 4	Floyd's triangle from 1 to 10	Pass
4	Choice 4, rows 3	Centered palindrome pyramid	Pass
5	Choice 5, rows 3	Symmetric diamond with 5 lines	Pass
6	Choice 9	Invalid menu choice message	Pass
7	Choice 1, rows 0	Row-count validation message	Pass

Output Screenshots:



Conclusion:

The Advanced Numeric Pattern Generator demonstrates the use of loops, nested loops, functions, conditional statements, input validation, and menu-driven programming in Python. It generates five different numeric patterns and handles invalid inputs without terminating unexpectedly. The project can be extended with additional patterns, colorized terminal output, or a graphical user interface.

4.Password Policy Validation Engine

Problem Statement:

Weak passwords can make user accounts vulnerable to unauthorized access. Manually checking every password against a set of security rules is inconsistent and time-consuming. The Password Policy Validation Engine is a Python program that checks a password against a defined policy and reports whether it is valid. If the password fails, the program lists the rules that were not satisfied.

Objectives:

Validate passwords automatically using a consistent set of rules.

Require a minimum password length of 8 characters.

Check for uppercase letters, lowercase letters, digits, and special characters.

Reject passwords containing whitespace.

Display clear validation feedback without revealing the entered password on screen.

Make policy settings easy to modify for future requirements.

Assumptions:

Python 3 is installed on the computer.

The program runs in a terminal or command prompt.

The default policy requires at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one special character.

Whitespace characters are not allowed.

The program validates a password locally and does not store or transmit it.

This is an educational demonstration; production systems should also use secure password storage, rate limiting, and other account-security controls.

Algorithm:

1. Start the program.

2. Display the password policy.

3. Read the password using hidden terminal input.

4. Create an empty list of validation errors.

5. Check the password length.

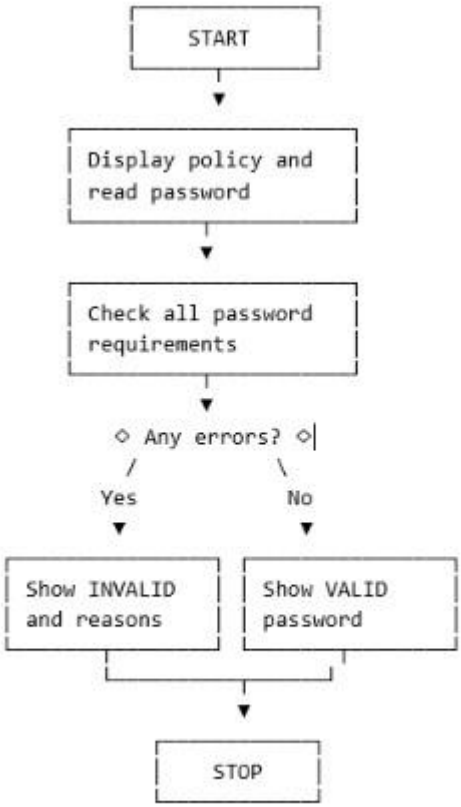
6. Check uppercase, lowercase, digit, and special-character requirements.

7. Check whether the password contains whitespace.

8. If no errors exist, display VALID; otherwise, display INVALID and list each error.

9. Stop the program.

Flowchart:



Pseudocode:

```
BEGIN
  SET minimum_length = 8
  INPUT password (hide input)
  SET errors = empty list

  IF length(password) < minimum_length
    ADD minimum-length error
  IF password has no uppercase letter
    ADD uppercase error
  IF password has no lowercase letter
    ADD lowercase error
  IF password has no digit
```

```
    ADD digit error
  IF password has no special character
    ADD special-character error
  IF password contains whitespace
    ADD whitespace error

  IF errors is empty
    DISPLAY "VALID password"
  ELSE
    DISPLAY "INVALID password"
    DISPLAY each error
END
```

Python Source Code:

```
import re
import getpass

MIN_LENGTH = 8
REQUIRE_UPPERCASE = True
REQUIRE_LOWERCASE = True
REQUIRE_DIGIT = True
REQUIRE_SPECIAL = True

SPECIAL_CHARACTERS = r"!@#$%^&*()-_=+[]{};:,.?/"

def validate_password(password):
  """Return (is_valid, errors) based on the configured password policy."""
  errors = []

  if len(password) < MIN_LENGTH:
    errors.append(f"Password must contain at least {MIN_LENGTH} characters.")
  if REQUIRE_UPPERCASE and not re.search(r"[A-Z]", password):
    errors.append("Include at least one uppercase letter (A-Z).")
  if REQUIRE_LOWERCASE and not re.search(r"[a-z]", password):
    errors.append("Include at least one lowercase letter (a-z).")
  if REQUIRE_DIGIT and not re.search(r"\d", password):
    errors.append("Include at least one digit (0-9).")
  if REQUIRE_SPECIAL and not any(ch in SPECIAL_CHARACTERS for ch in password):
    errors.append("Include at least one special character.")
  if re.search(r"\s", password):
    errors.append("Password must not contain whitespace.")
```

```
    return len(errors) == 0, errors

def main():
    print("=== Password Policy Validation Engine ===")
    print(f"Policy: minimum {MIN_LENGTH} characters; uppercase, lowercase, digit,")
    print("special character required; whitespace is not allowed.")
    password = getpass.getpass("Enter password (input hidden): ")

    is_valid, errors = validate_password(password)
    if is_valid:
        print("Result: VALID password.")
    else:
        print("Result: INVALID password.")
        print("Reasons:")
        for error in errors:
            print(f"- {error}")

if __name__ == "__main__":
    main()
```

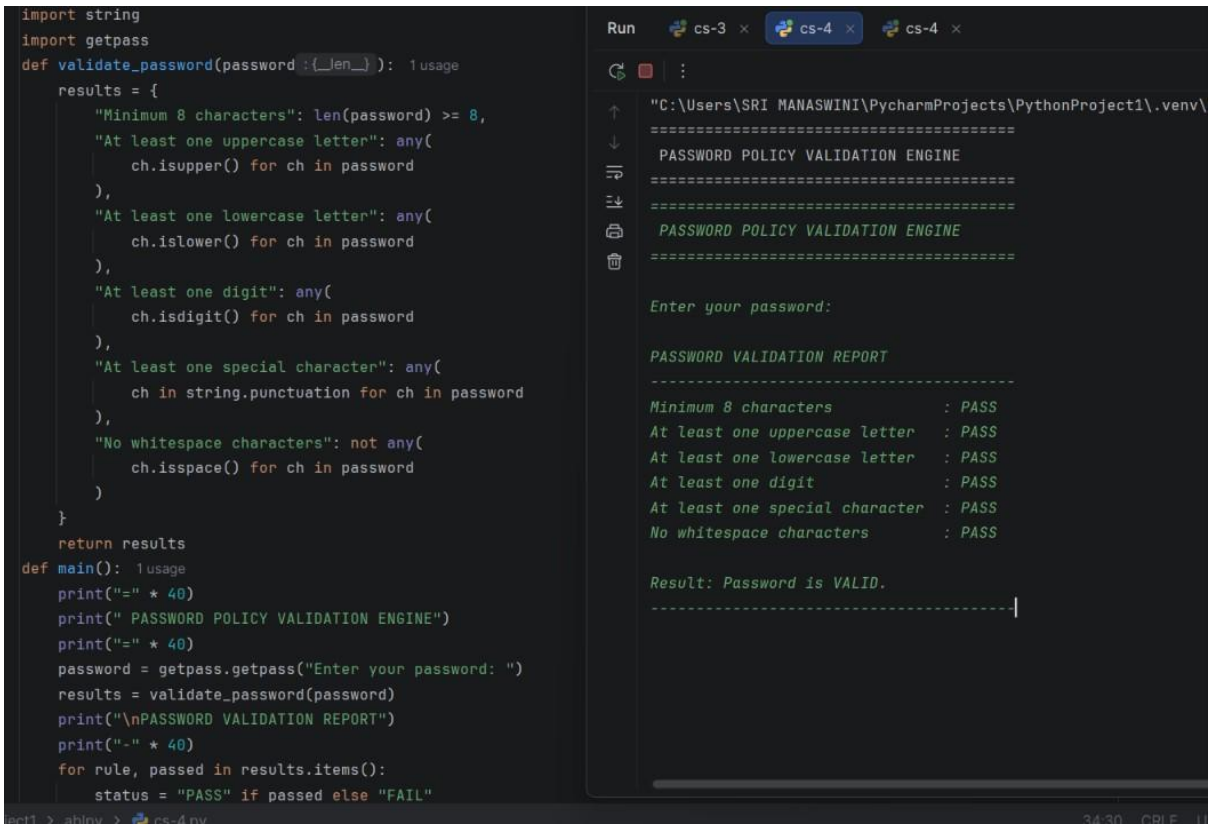
Test Cases:

Test ID	Input	Expected result	Status
T01	Abcdef1!	Valid: all rules satisfied	PASS
T02	abc1!def	Invalid: missing uppercase	PASS
T03	ABCDEFG1!	Invalid: missing lowercase	PASS
T04	Abcdefgh!	Invalid: missing digit	PASS
T05	Abcdefg1	Invalid: missing special character	PASS
T06	A1!bc	Invalid: shorter than 8 characters	PASS

T07	Abcd 1!ef	Invalid: contains whitespace	PASS
T08	P@ssw0rd2026	Valid: all rules satisfied	PASS
T09		Invalid: empty password; multiple rules fail	PASS

Note: Test passwords are examples only. Do not reuse them for real accounts.

Sample Output Screenshot:



Conclusion:

The Password Policy Validation Engine demonstrates how Python can automate password-policy checks. It checks length, character categories, and whitespace, then provides actionable feedback. The code is modular and the policy constants can be adjusted as requirements change. For real-world deployment, password validation should be integrated with secure authentication practices and passwords must never be stored in plain text.

5.Smart Parking Fee Management

Problem Statement:

Managing parking fees manually can lead to calculation errors, inconsistent charges, and delays at the exit. The Smart Parking Fee Management System calculates the parking fee based on the vehicle type and the duration of parking. It accepts vehicle details and parking duration, applies the fee rules, displays an itemized summary, and validates incorrect input.

Objectives:

Calculate parking charges automatically using vehicle type and parking duration.

Apply a minimum one-hour charge and charge each started hour as a full hour.

Support cars, bikes, and buses with different hourly rates.

Validate user input and show a clear fee receipt.

Demonstrate the solution using an algorithm, flowchart, pseudocode, Python code, and test cases

Algorithm:

Start the program.

Display the supported vehicle types and hourly rates.

Read the vehicle type and parking duration from the user.

Normalize the vehicle type to make the comparison case-insensitive.

Check whether the vehicle type is supported and the duration is a positive number.

Select the hourly rate for the chosen vehicle type.

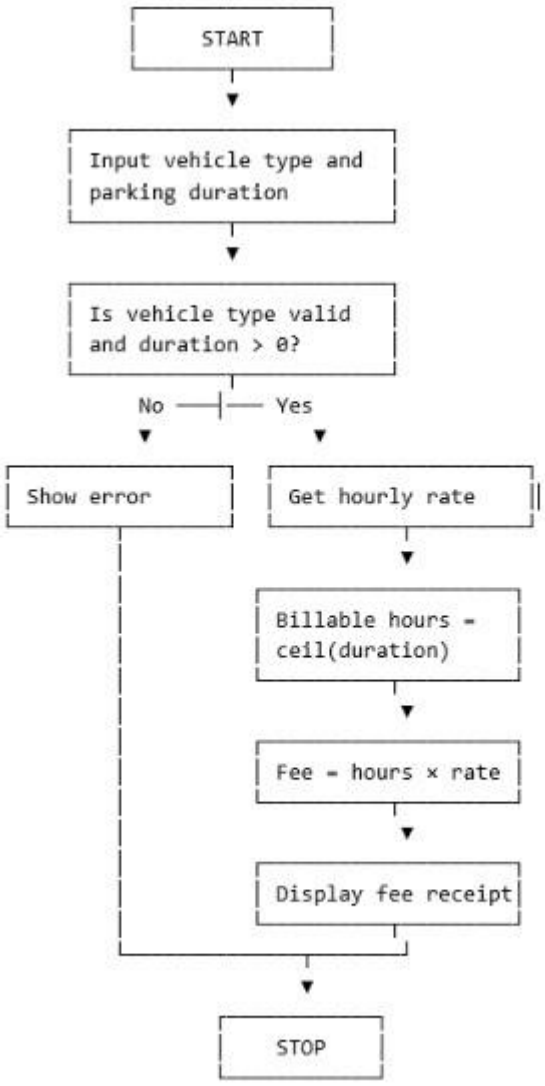
Calculate billable hours by rounding the duration up to the next whole hour.

Calculate the total fee as billable hours multiplied by the hourly rate.

Display the vehicle type, duration, billable hours, rate, and total fee.

Stop the program

Flowchart:



Pseudocode:

```
BEGIN
  rates ← {"bike": 20, "car": 50, "bus": 100}

  INPUT vehicle_type
  INPUT duration

  vehicle_type ← lowercase(trim(vehicle_type))

  IF vehicle_type NOT IN rates THEN
    PRINT "Invalid vehicle type"
  ELSE IF duration is not numeric OR duration <= 0 THEN
    PRINT "Parking duration must be a positive number"
  ELSE
    billable_hours ← ceiling(duration)
    hourly_rate ← rates[vehicle_type]
```

```
        total_fee ← billable_hours × hourly_rate

        PRINT vehicle type
        PRINT duration
        PRINT billable_hours
        PRINT hourly_rate
        PRINT total_fee
    END IF
END
```

Python Source Code:

```
import math

HOURLY_RATES = {
    "bike": 20,
    "car": 50,
    "bus": 100
}

def calculate_parking_fee(vehicle_type, duration):
    """Return billable hours, hourly rate, and total parking fee."""
    vehicle = vehicle_type.strip().lower()

    if vehicle not in HOURLY_RATES:
        raise ValueError("Invalid vehicle type. Choose Bike, Car, or Bus.")

    try:
        hours = float(duration)
    except (TypeError, ValueError):
        raise ValueError("Parking duration must be a number.")

    if not math.isfinite(hours) or hours <= 0:
        raise ValueError("Parking duration must be a positive number.")

    billable_hours = math.ceil(hours)
    hourly_rate = HOURLY_RATES[vehicle]
    total_fee = billable_hours * hourly_rate

    return vehicle.title(), hours, billable_hours, hourly_rate, total_fee


def main():
    print("SMART PARKING FEE MANAGEMENT")
```

```
    print("Rates: Bike ₹20/hour | Car ₹50/hour | Bus ₹100/hour")

    vehicle_type = input("Enter vehicle type (Bike/Car/Bus): ")
    duration = input("Enter parking duration in hours: ")

    try:
        vehicle, actual_hours, billable_hours, rate, fee = (
            calculate_parking_fee(vehicle_type, duration)
        )
    except ValueError as error:
        print("Error:", error)
        return

    print("\n----- PARKING FEE RECEIPT -----")
    print(f"Vehicle type   : {vehicle}")
    print(f"Actual duration: {actual_hours:g} hour(s)")
    print(f"Billable hours : {billable_hours}")
    print(f"Rate per hour  : ₹{rate}")
    print(f"Total fee      : ₹{fee}")
    print("-------------------------------")

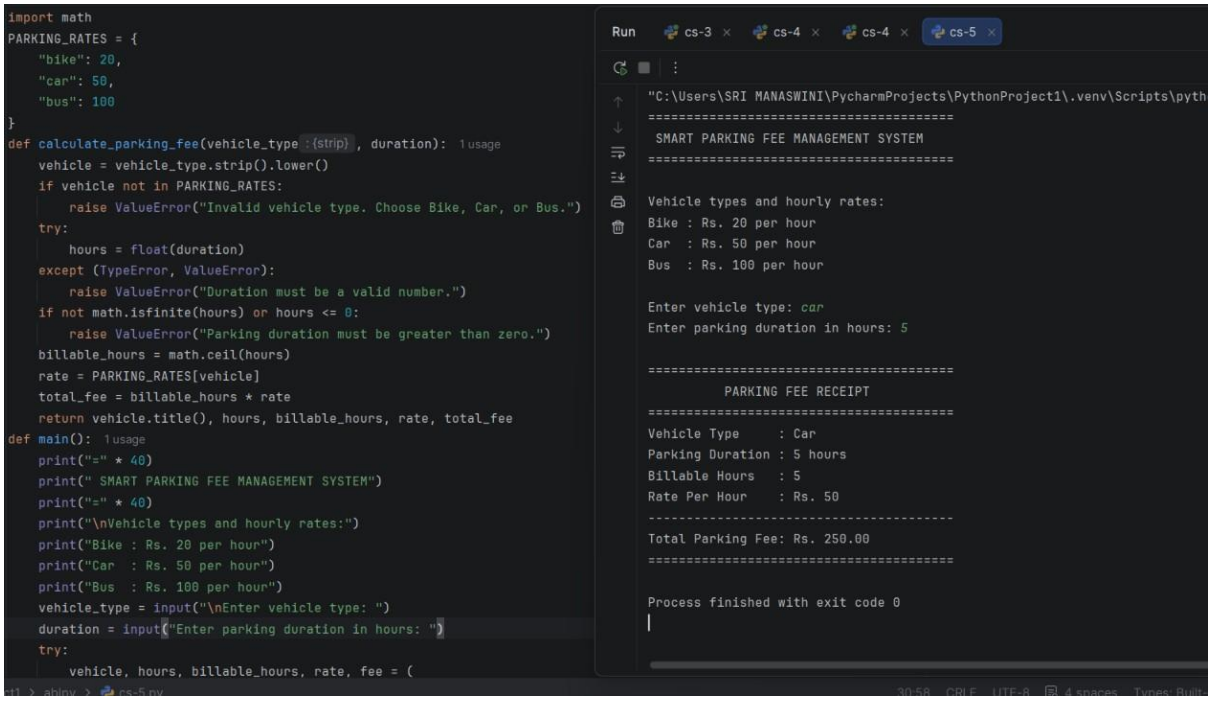

if __name__ == "__main__":
    main()
```

Test Cases:

Test ID	Input type	Duration (h)	Expected result	Billable hours	Expected fee	Purpose
TC01	Bike	1	Bike	1	₹20	Valid minimum one-hour charge
TC02	Car	2	Car	2	₹100	Valid whole-hour duration
TC03	Bus	2.5	Bus	3	₹300	Partial hour rounds up
TC04	Car	0.25	Car	1	₹50	Less than one hour is

						billed as one hour
TC05	bike	1.2	Bike	2	₹40	Vehicle type is case-insensitive
TC06	Truck	2	Error	—	—	Unsupported vehicle type rejected
TC07	Bike	0	Error	—	—	Zero duration rejected
TC08	Car	-1	Error	—	—	Negative duration rejected
TC09	Bus	abc	Error	—	—	Non-numeric duration rejected

Sample Output Screenshot:



Conclusion:

The Smart Parking Fee Management System provides a simple and reliable way to calculate parking charges from vehicle type and parking duration. Rounding up partial hours ensures that the stated billing rule is applied consistently, while input

validation prevents invalid vehicle types and durations from producing incorrect fees. The program can be extended in future with vehicle entry/exit timestamps, parking-slot tracking, digital payments, discounts, and receipt storage

6.Multi-Day Water Tank Simulation

Problem Statement:

Water tanks store water for homes and buildings. Tracking the water level over several days helps identify shortages and overflow. This Python simulation calculates the daily closing water level using tank capacity, initial water level, daily inflow, and daily demand. It also reports overflow, unmet demand, and low-water warnings.

Objectives:

Simulate water levels for a selected number of days.

Account for daily inflow and consumption.

Prevent water levels from exceeding tank capacity or falling below zero.

Display daily opening level, inflow, supplied water, overflow, unmet demand, and closing level.

Show warnings for low water, shortages, and overflow.

Demonstrate Python loops, conditions, functions, and input validation.

Assumptions:

Capacity	**A fixed positive tank capacity measured in liters.**
Initial level	Must be between zero and tank capacity.
Daily inflow	A fixed non-negative quantity added each day.
Daily demand	A fixed non-negative quantity requested each day.
Overflow	Water above capacity is discarded and recorded.
Shortage	If demand exceeds available water, only available water is supplied; the remainder is unmet demand.
Low-water warning	Displayed when the closing level is at or below 20% of capacity.
Scope	A simplified educational simulation; it does not connect to real tank sensors.

Algorithm:

Start the program.

Read capacity, initial level, number of days, daily inflow, and daily demand.

Validate all inputs.

For each day, add inflow to the opening level.

Record and remove any water exceeding tank capacity.

Supply the smaller of demand and available water.

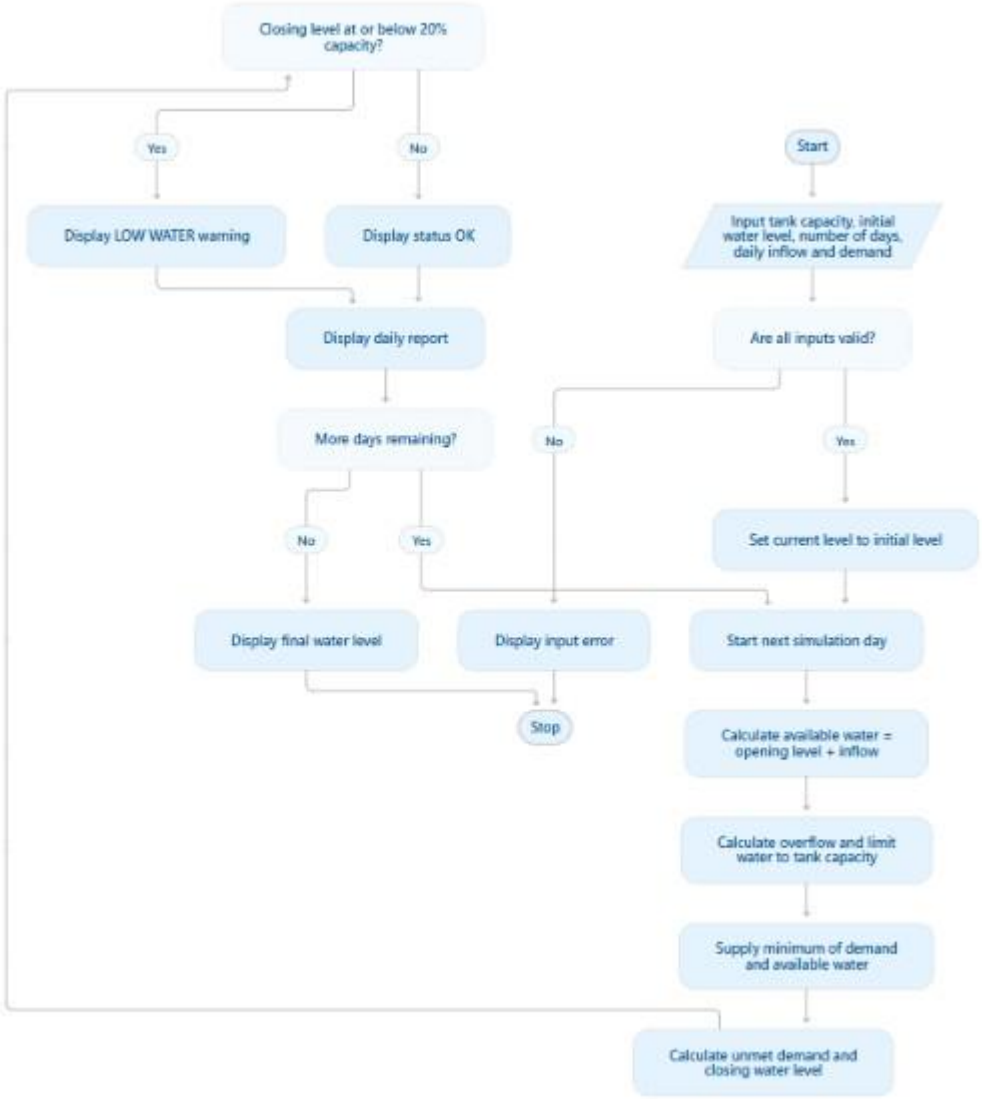
Calculate unmet demand and closing water level.

Display the daily report and low-water warning if required.

Use the closing level as the next day's opening level.

Stop after all days have been simulated.

Flowchart:



Pseudocode:

```
BEGIN
   INPUT capacity, initial_level, days, daily_inflow, daily_demand
   VALIDATE inputs
   level ← initial_level

   FOR day ← 1 TO days
      opening ← level
      available ← opening + daily_inflow
      overflow ← MAX(available - capacity, 0)
      available ← MIN(available, capacity)
      supplied ← MIN(daily_demand, available)
      unmet ← daily_demand - supplied
      level ← available - supplied
      DISPLAY daily values and level
```

```
        IF level <= 0.20 * capacity
            DISPLAY "LOW WATER WARNING"
        END IF
    END FOR
END
```

Python Source Code:

```
def simulate_water_tank(capacity, initial_level, days,
                        daily_inflow, daily_demand):
    if capacity <= 0:
        raise ValueError("Capacity must be greater than zero.")
    if not 0 <= initial_level <= capacity:
        raise ValueError("Initial level must be between 0 and capacity.")
    if days <= 0:
        raise ValueError("Days must be greater than zero.")
    if daily_inflow < 0 or daily_demand < 0:
        raise ValueError("Inflow and demand cannot be negative.")

    level = initial_level
    records = []

    for day in range(1, days + 1):
        opening = level
        available = opening + daily_inflow
        overflow = max(available - capacity, 0)
        available = min(available, capacity)
        supplied = min(daily_demand, available)
        unmet = daily_demand - supplied
        level = available - supplied
        status = "LOW WATER" if level <= 0.20 * capacity else "OK"

        records.append((day, opening, daily_inflow, supplied,
                        overflow, unmet, level, status))

    return records


def read_nonnegative(prompt):
    value = float(input(prompt))
    if value < 0:
        raise ValueError("Value cannot be negative.")
    return value
```

```
def main():
    print("MULTI-DAY WATER TANK SIMULATION")
    try:
        capacity = read_nonnegative("Tank capacity (liters): ")
        if capacity == 0:
            raise ValueError("Capacity must be greater than zero.")

        initial = read_nonnegative("Initial water level (liters): ")
        days = int(input("Number of days: "))
        inflow = read_nonnegative("Daily inflow (liters): ")
        demand = read_nonnegative("Daily water demand (liters): ")

        records = simulate_water_tank(
            capacity, initial, days, inflow, demand
        )

        print("\nDAILY SIMULATION REPORT")
        print("Day | Opening | Inflow | Used | Overflow | Unmet | Closing | Status")
        for r in records:
            day, opening, inflow, used, overflow, unmet, closing, status = r
            print(f"{day:>3} | {opening:>7.1f} | {inflow:>6.1f} | "
                  f"{used:>4.1f} | {overflow:>8.1f} | {unmet:>5.1f} | "
                  f"{closing:>7.1f} | {status}")

        print(f"\nFinal water level: {records[-1][6]:.1f} liters")

    except ValueError as error:
        print("Input error:", error)


if __name__ == "__main__":
    main()
```

Test Cases:

ID	Input (capacity, initial, days, inflow, demand)	Expected result	Purpose
TC01	1000, 500, 3, 100, 150	Closing levels: 450, 400, 350 L	Normal simulation

TC02	1000, 950, 2, 100, 50	Day 1 overflow = 50 L; closing = 950 L	Overflow
TC03	500, 100, 2, 0, 80	Day 1 closes at 20 L; Day 2 unmet demand = 60 L	Shortage
TC04	1000, 100, 2, 50, 100	Day 1 closes at 50 L and shows LOW WATER	Low-level warning
TC05	0, 0, 2, 10, 10	Error: capacity must be positive	Invalid capacity
TC06	100, 120, 1, 0, 10	Error: initial level exceeds capacity	Invalid initial level
TC07	100, 50, 0, 0, 10	Error: days must be positive	Invalid days
TC08	100, 50, 2, -10, 10	Error: inflow cannot be negative	Invalid inflow

Sample Output:

```
def simulate_water_tank(capacity, initial_level, days,
                        daily_inflow, daily_demand):
    if capacity <= 0:
        raise ValueError("Capacity must be greater than zero.")
    if not 0 <= initial_level <= capacity:
        raise ValueError("Initial level must be within tank capacity.")
    if days <= 0:
        raise ValueError("Number of days must be positive.")
    if daily_inflow < 0 or daily_demand < 0:
        raise ValueError("Inflow and demand cannot be negative.")
    level = initial_level
    print("\nMULTI-DAY WATER TANK SIMULATION")
    print("-" * 75)
    print("Day | Opening | Inflow | Supplied | Overflow | Unmet | Closing | Stat
    print("-" * 75)
    for day in range(1, days + 1):
        opening = level
        available = opening + daily_inflow
        overflow = max(available - capacity, 0)
        available = min(available, capacity)
        supplied = min(daily_demand, available)
        unmet = daily_demand - supplied
        level = available - supplied
        status = "LOW WATER" if level <= capacity * 0.20 else "OK"
        print(
            f"{day:3} | {opening:7.1f} | {daily_inflow:6.1f} | "
            f"{supplied:8.1f} | {overflow:8.1f} | {unmet:5.1f} | "
            f"{level:7.1f} | {status}"
        )
    print("-" * 75)
    print(f"Final water level: {level:.1f} liters")
```

```
Run   cs-3 × cs-4 × cs-4 × cs-6 ×

Enter tank capacity (liters): 5
Enter initial water level (liters): 3
Enter number of days: 12
Enter daily inflow (liters): 21
Enter daily water demand (liters): 4

MULTI-DAY WATER TANK SIMULATION
---------------------------------------------------------------------------
Day | Opening | Inflow | Supplied | Overflow | Unmet | Closing | Status
---------------------------------------------------------------------------
  1 |     3.0 |   21.0 |      4.0 |     19.0 |   0.0 |     1.0 | LOW WATER
  2 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  3 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  4 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  5 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  6 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  7 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  8 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
  9 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
 10 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
 11 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
 12 |     1.0 |   21.0 |      4.0 |     17.0 |   0.0 |     1.0 | LOW WATER
---------------------------------------------------------------------------
Final water level: 1.0 liters

Process finished with exit code 0
```

Conclusion

The Multi-Day Water Tank Simulation tracks stored water across multiple days while accounting for inflow, demand, overflow, unmet demand, and low-water warnings. It demonstrates Python functions, loops, conditions, input validation, and data

handling. Future improvements could include variable daily demand, rainfall, leakage, sensor readings, and graphs.

7.Student Performance Trend Analysis

Problem Statement:

Students receive marks in different subjects across multiple examinations. Reviewing each mark separately can make it difficult to understand whether a student is improving, declining, or maintaining a similar level of performance. This project uses Python to record marks for several exams, calculate average scores and percentages, classify performance, and compare consecutive exam averages to identify a simple performance trend.

Objectives:

Collect student name, subject names, and marks from multiple examinations.

Validate marks so they are within the permitted range of 0 to 100.

Calculate each examination average and overall average.

Calculate the overall percentage and assign a performance category.

Compare exam averages to identify an improving, declining, or stable trend.

Display a clear report that can help students and teachers review progress.

Assumptions:

Marks range	**Each subject mark is a number from 0 to 100 inclusive.**
Examinations	At least two examinations are entered so that a trend can be compared.
Subjects	The same subjects are assessed in each examination.
Exam averages	Each subject is equally weighted when calculating an examination average.
Trend rule	If the latest average exceeds the previous average by more than 1 point, the trend is Improving; if it is lower by more than 1 point, it is Declining; otherwise it is Stable.

Performance categories	Excellent: 90–100; Very Good: 75–89.99; Good: 60–74.99; Average: 40–59.99; Needs Improvement: below 40.
Scope	The program uses user-entered marks and does not connect to a school database.

Algorithm:

Start the program.

Read the student name, number of subjects, subject names, and number of examinations.

For each examination, input the marks for every subject.

Validate that each mark is numeric and between 0 and 100.

Calculate the average mark for each examination.

Calculate the overall average and percentage across all entered marks.

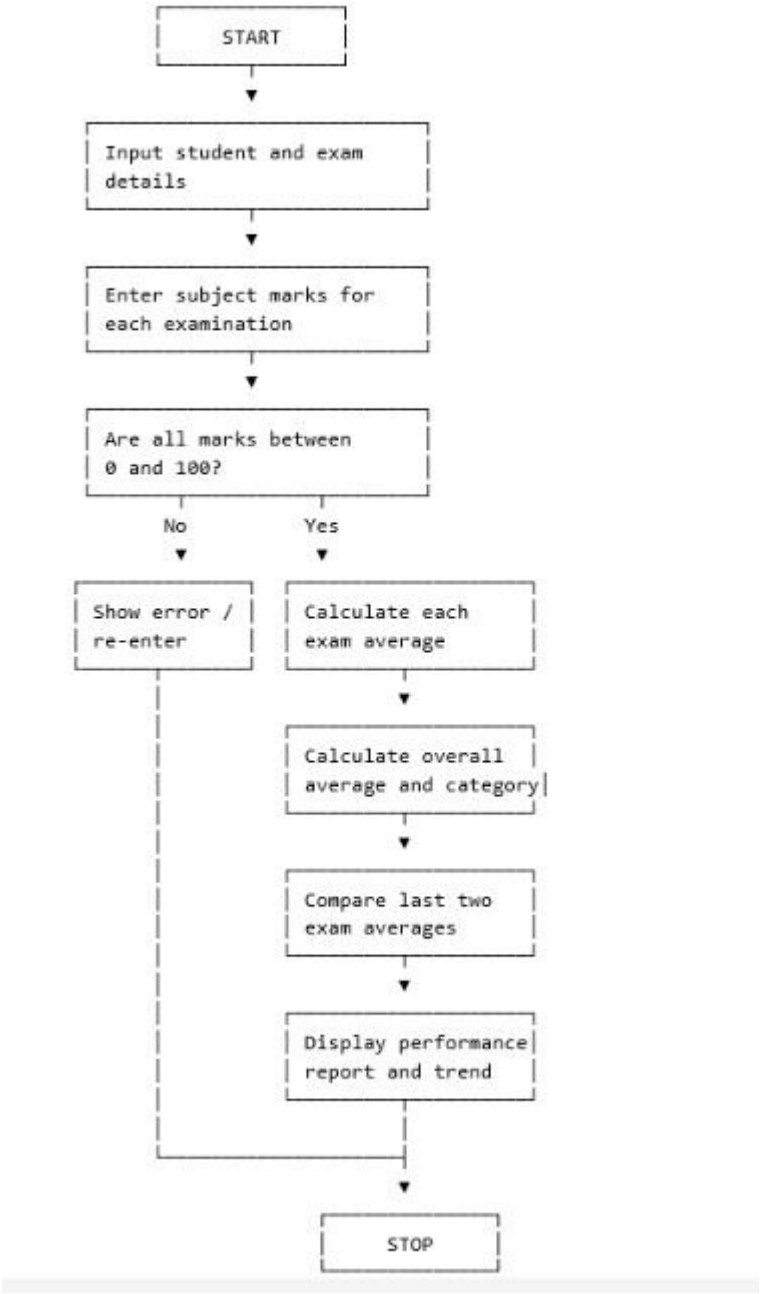
Assign a performance category based on the overall average.

Compare the latest examination average with the previous examination average.

Display the examination averages, overall result, category, and trend.

Stop the program.

Flowchart:



Pseudocode:

```
BEGIN
  INPUT student_name
  INPUT subject_names
  INPUT number_of_exams (must be at least 2)

  FOR each exam
    FOR each subject
      INPUT mark
      WHILE mark < 0 OR mark > 100
        DISPLAY error and INPUT mark again
```

```
      END WHILE
    END FOR
    exam_average ← SUM(exam_marks) / number_of_subjects
  END FOR

  overall_average ← SUM(all_marks) / total_number_of_marks
  percentage ← overall_average
  category ← classify(overall_average)

  difference ← latest_exam_average - previous_exam_average
  IF difference > 1
    trend ← "Improving"
  ELSE IF difference < -1
    trend ← "Declining"
  ELSE
    trend ← "Stable"
  END IF

  DISPLAY exam averages, overall average, category, trend
END
```

Python Source Code:

```
def performance_category(average):
  if average >= 90:
    return "Excellent"
  elif average >= 75:
    return "Very Good"
  elif average >= 60:
    return "Good"
  elif average >= 40:
    return "Average"
  return "Needs Improvement"


def performance_trend(previous_average, latest_average):
  difference = latest_average - previous_average

  if difference > 1:
    return "Improving"
  elif difference < -1:
    return "Declining"
```

```
    return "Stable"


def main():
    print("=" * 45)
    print("      STUDENT PERFORMANCE TREND ANALYSIS")
    print("=" * 45)

    name = input("Enter student name: ").strip()
    if not name:
        print("Error: Student name cannot be empty.")
        return

    try:
        subject_count = int(input("Number of subjects: "))
        exam_count = int(input("Number of examinations (minimum 2): "))

        if subject_count <= 0:
            raise ValueError("Number of subjects must be positive.")
        if exam_count < 2:
            raise ValueError("Enter at least two examinations.")

        subjects = []
        for i in range(subject_count):
            subject = input(f"Enter subject {i + 1} name: ").strip()
            if not subject:
                raise ValueError("Subject name cannot be empty.")
            subjects.append(subject)

        exam_marks = []
        exam_averages = []

        for exam in range(exam_count):
            print(f"\nEnter marks for Examination {exam + 1}")
            marks = []

            for subject in subjects:
                while True:
                    try:
                        mark = float(input(f"{subject} (0-100): "))
                        if not 0 <= mark <= 100:
```

```
                print("Marks must be between 0 and 100.")
                continue
              marks.append(mark)
              break
            except ValueError:
              print("Please enter a valid number.")

        exam_marks.append(marks)
        exam_averages.append(sum(marks) / len(marks))

      all_marks = [mark for exam in exam_marks for mark in exam]
      overall_average = sum(all_marks) / len(all_marks)
      overall_percentage = overall_average  # Marks are out of 100.

      print("\n" + "=" * 45)
      print("          PERFORMANCE REPORT")
      print("=" * 45)
      print(f"Student Name      : {name}")

      for i, average in enumerate(exam_averages, start=1):
        print(f"Examination {i} Avg : {average:.2f}/100")

      print("-" * 45)
      print(f"Overall Average    : {overall_average:.2f}/100")
      print(f"Overall Percentage : {overall_percentage:.2f}%")
      print(f"Performance       : {performance_category(overall_average)}")
      print(
        "Latest Trend      : "
        + performance_trend(exam_averages[-2], exam_averages[-1])
      )
      print("=" * 45)

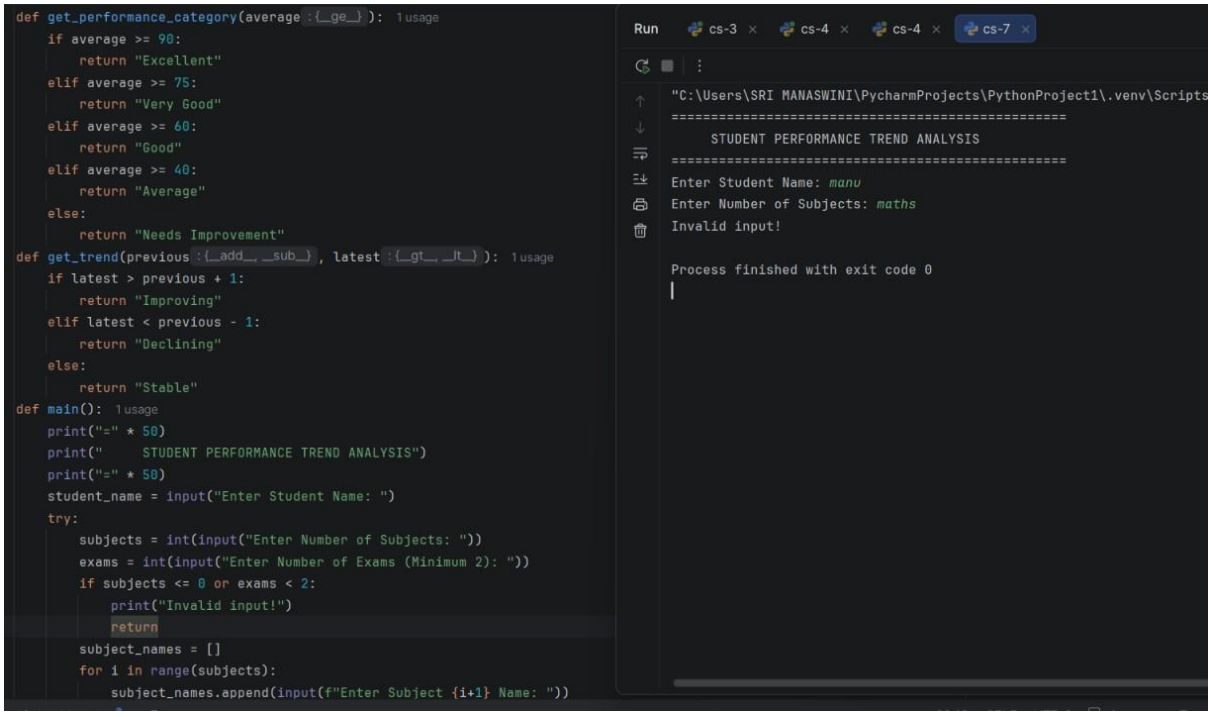
    except ValueError as error:
      print("Input error:", error)


if __name__ == "__main__":
    main()
```

Test Cases:

Test ID	Exam averages / input condition	Expected result	Purpose
TC01	Exam 1: 70, Exam 2: 80	Overall average 75; trend Improving	Improvement
TC02	Exam 1: 85, Exam 2: 75	Overall average 80; trend Declining	Decline
TC03	Exam 1: 70, Exam 2: 70.5	Trend Stable	Small difference
TC04	All marks 90	Category Excellent	High performance
TC05	All marks 50	Category Average	Average performance
TC06	Mark -5 or 105	Reject mark and ask for valid mark	Mark validation
TC07	Only one examination	Input error: at least two examinations	Exam validation
TC08	No student name	Error: name cannot be empty	Name validation

Sample Output Screenshot:



```
def get_performance_category(average: float) -> str:
    if average >= 90:
        return "Excellent"
    elif average >= 75:
        return "Very Good"
    elif average >= 60:
        return "Good"
    elif average >= 40:
        return "Average"
    else:
        return "Needs Improvement"
def get_trend(previous: int, latest: int) -> str:
    if latest > previous + 1:
        return "Improving"
    elif latest < previous - 1:
        return "Declining"
    else:
        return "Stable"
def main():
    print("=" * 50)
    print("     STUDENT PERFORMANCE TREND ANALYSIS")
    print("=" * 50)
    student_name = input("Enter Student Name: ")
    try:
        subjects = int(input("Enter Number of Subjects: "))
        exams = int(input("Enter Number of Exams (Minimum 2): "))
        if subjects <= 0 or exams < 2:
            print("Invalid input!")
            return
        subject_names = []
        for i in range(subjects):
            subject_names.append(input(f"Enter Subject {i+1} Name: "))
```

```
Run  cs-3 × cs-4 × cs-4 × cs-7 ×
"C:\Users\SRI MANASWINI\PycharmProjects\PythonProject1\.venv\Scripts
==================================================
     STUDENT PERFORMANCE TREND ANALYSIS
==================================================
Enter Student Name: manu
Enter Number of Subjects: maths
Invalid input!

Process finished with exit code 0
```

Conclusion:

The Student Performance Trend Analysis program summarizes marks across multiple examinations and identifies whether the latest performance is improving, declining, or stable. It calculates exam averages, an overall average, percentage, and a performance category. The project demonstrates Python functions, loops, lists, conditions, and input validation. It could be extended with graphs, file storage, subject-wise trends, and comparisons among multiple students.

8.Traffic Violation Penalty Calculator

Problem Statement:

Traffic violations may result in penalties that vary according to the type and number of violations. Calculating a total manually can cause mistakes. This project develops a Python program that accepts one or more traffic violation types, calculates an illustrative penalty for each, and displays an itemized summary and total. The program validates inputs and can apply a simple repeat-violation multiplier.

Objectives:

Provide a menu of traffic violation categories.

Calculate the penalty for each selected violation.

Support multiple violations in one calculation.

Optionally apply a repeat-violation multiplier.

Validate menu choices and the number of repeat offences.

Display an itemized penalty report and total amount.

Assumptions:

Currency	**Amounts are shown in Indian rupees (₹).**
Illustrative rates	Overspeeding ₹1,000; no helmet ₹500; seat-belt violation ₹500; signal violation ₹1,000; illegal parking ₹500; using a mobile phone while driving ₹1,500.
Not legal advice	These amounts are example values for a programming exercise, not a statement of current official fines. Actual penalties depend on jurisdiction and applicable law.
Multiple violations	Each selected violation is charged separately and all penalties are added.
Repeat violations	The user enters a repeat count. If it is greater than one, the program applies a configurable 1.5 multiplier to the total as a simplified example.

Input limits	At least one violation must be selected; repeat count must be a positive integer.
Scope	The program does not access government records, identify vehicles, or issue real challans.

Algorithm:

Start the program.

Display the list of violation types and example penalties.

Ask the user to select one or more violations.

Validate each selected option and add its penalty to the subtotal.

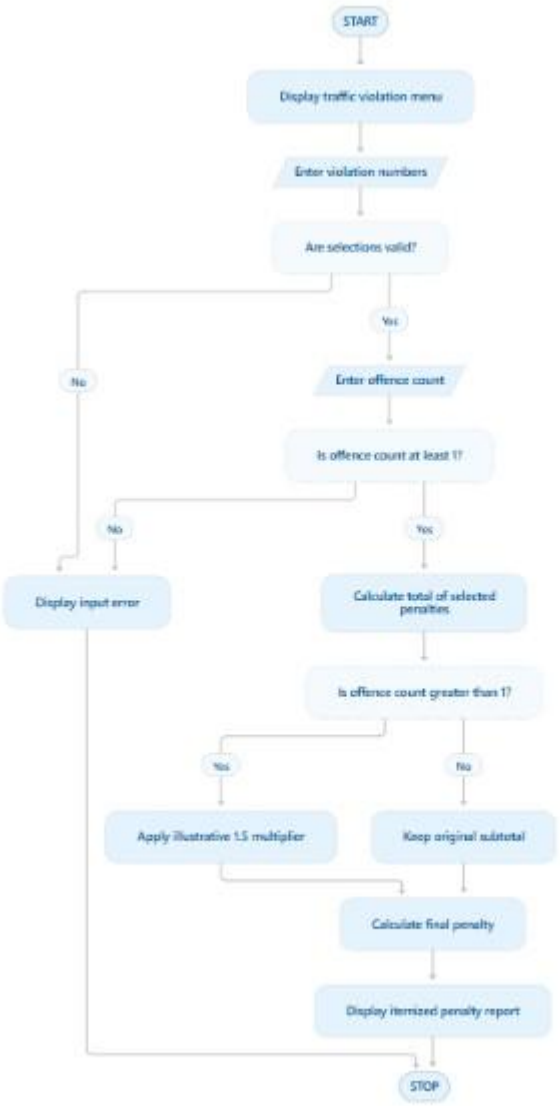
Ask for the repeat-offence count and validate that it is a positive integer.

If the repeat count is greater than one, apply the example repeat multiplier.

Display each selected violation, its penalty, subtotal, any multiplier, and final total.

Stop the program.

Flowchart:



Pseudocode:

```
BEGIN
  DEFINE violation names and example penalty amounts
  DISPLAY violation menu
  INPUT selected violation numbers
  VALIDATE that all selections are valid

  subtotal ← 0
  FOR each selected violation
    subtotal ← subtotal + its penalty
  END FOR

  INPUT repeat_count
  VALIDATE repeat_count >= 1
```

```
  IF repeat_count > 1
    final_total ← subtotal × 1.5
  ELSE
    final_total ← subtotal
  END IF

  DISPLAY itemized penalties, subtotal, repeat count, final_total
END
```

Python Source Code:

```
VIOLATIONS = {
  1: ("Overspeeding", 1000),
  2: ("Riding without a helmet", 500),
  3: ("Driving without a seat belt", 500),
  4: ("Traffic signal violation", 1000),
  5: ("Illegal parking", 500),
  6: ("Using a mobile phone while driving", 1500),
}

REPEAT_MULTIPLIER = 1.5  # Illustrative project rule, not an official fine rule


def calculate_penalty(selected_ids, repeat_count):
  if not selected_ids:
    raise ValueError("Select at least one violation.")
  if any(item not in VIOLATIONS for item in selected_ids):
    raise ValueError("One or more violation choices are invalid.")
  if repeat_count < 1:
    raise ValueError("Repeat count must be at least 1.")

  subtotal = sum(VIOLATIONS[item][1] for item in selected_ids)
  multiplier = REPEAT_MULTIPLIER if repeat_count > 1 else 1.0
  total = subtotal * multiplier
  return subtotal, multiplier, total


def main():
  print("=" * 50)
  print("      TRAFFIC VIOLATION PENALTY CALCULATOR")
  print("=" * 50)
```

```
    print("Example rates for educational use only. Verify actual local fines.\n")

    for number, (name, amount) in VIOLATIONS.items():
        print(f"{number}. {name}: Rs. {amount}")

    try:
        raw = input("\nEnter violation numbers separated by commas (e.g. 1,3): ")
        selected_ids = [int(part.strip()) for part in raw.split(",") if part.strip()]

        repeat_count = int(input("Enter offence count (1 = first offence): "))

        subtotal, multiplier, total = calculate_penalty(
            selected_ids, repeat_count
        )

        print("\n" + "-" * 50)
        print("            PENALTY REPORT")
        print("-" * 50)

        for item in selected_ids:
            name, amount = VIOLATIONS[item]
            print(f"{name:<36} Rs. {amount:>7.2f}")

        print("-" * 50)
        print(f"{'Subtotal':<36} Rs. {subtotal:>7.2f}")
        if multiplier > 1:
            print(f"Repeat-offence multiplier: x{multiplier:.1f} (example rule)")
        print(f"{'TOTAL PENALTY':<36} Rs. {total:>7.2f}")
        print("-" * 50)
        print("Note: Example calculations only; check applicable law.")

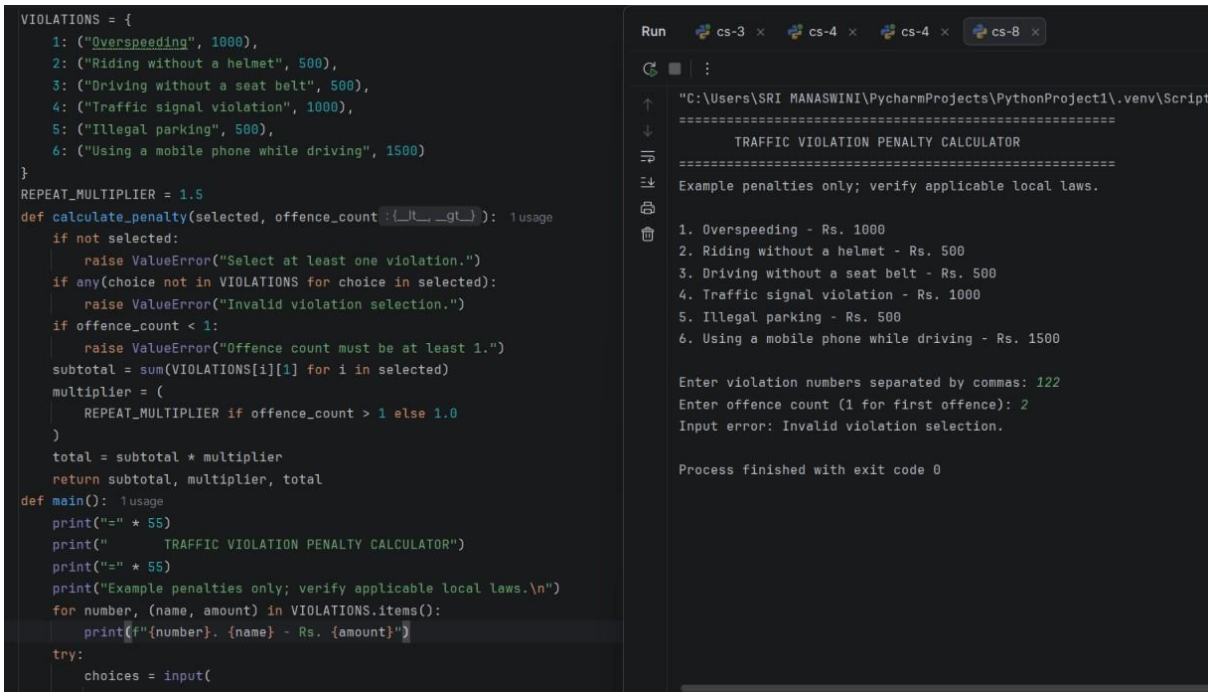
    except ValueError as error:
        print("Input error:", error)


if __name__ == "__main__":
    main()
```

Test Cases:

Test ID	Input	Expected result	Purpose
TC01	Violation 1; offence count 1	Subtotal ₹1,000; total ₹1,000	Single violation
TC02	Violations 2,3; offence count 1	Subtotal ₹1,000; total ₹1,000	Multiple violations
TC03	Violation 6; offence count 2	Subtotal ₹1,500; example total ₹2,250	Repeat multiplier
TC04	Choice 9	Input error for invalid choice	Invalid selection
TC05	No selection	Input error: select at least one violation	Empty selection
TC06	Offence count 0	Input error: count must be at least 1	Invalid repeat count
TC07	Input abc	Input error for non-numeric selection	Invalid data type

Sample Output Screenshot :



```
VIOLATIONS = {
    1: ("Overspeeding", 1000),
    2: ("Riding without a helmet", 500),
    3: ("Driving without a seat belt", 500),
    4: ("Traffic signal violation", 1000),
    5: ("Illegal parking", 500),
    6: ("Using a mobile phone while driving", 1500)
}
REPEAT_MULTIPLIER = 1.5
def calculate_penalty(selected, offence_count=1, _lt_=None, _gt_=None):  1 usage
    if not selected:
        raise ValueError("Select at least one violation.")
    if any(choice not in VIOLATIONS for choice in selected):
        raise ValueError("Invalid violation selection.")
    if offence_count < 1:
        raise ValueError("Offence count must be at least 1.")
    subtotal = sum(VIOLATIONS[i][1] for i in selected)
    multiplier = (
        REPEAT_MULTIPLIER if offence_count > 1 else 1.0
    )
    total = subtotal * multiplier
    return subtotal, multiplier, total
def main():  1 usage
    print("=" * 55)
    print("       TRAFFIC VIOLATION PENALTY CALCULATOR")
    print("=" * 55)
    print("Example penalties only; verify applicable local laws.\n")
    for number, (name, amount) in VIOLATIONS.items():
        print(f"{number}. {name} - Rs. {amount}")
    try:
        choices = input(
```

```
Run   cs-3 ×   cs-4 ×   cs-4 ×   cs-8 ×
"C:\Users\SRI MANASWINI\PycharmProjects\PythonProject1\.venv\Script
=======================================================
       TRAFFIC VIOLATION PENALTY CALCULATOR
=======================================================
Example penalties only; verify applicable local laws.

1. Overspeeding - Rs. 1000
2. Riding without a helmet - Rs. 500
3. Driving without a seat belt - Rs. 500
4. Traffic signal violation - Rs. 1000
5. Illegal parking - Rs. 500
6. Using a mobile phone while driving - Rs. 1500

Enter violation numbers separated by commas: 122
Enter offence count (1 for first offence): 2
Input error: Invalid violation selection.

Process finished with exit code 0
```

Conclusion:

The Traffic Violation Penalty Calculator demonstrates how Python can automate an itemized penalty calculation. It supports multiple violation types, validates user input, applies a clearly identified illustrative repeat-offence rule, and prints a summary. The example penalty amounts

and multiplier are for learning purposes only and must not be treated as current official fines. The project can be extended with jurisdiction-specific rules, date-based rates, vehicle details, and a database.

9.University Scholarship Allocation

Problem Statement:

A university receives scholarship applications from multiple students but has a limited fund. Manually checking eligibility and allocating funds can be time-consuming and inconsistent. This project automates a simplified allocation process using sample rules that can be adjusted to match an institution's official scholarship policy

Objectives:

Identify students who meet the scholarship eligibility requirements.

Use academic performance and family income as allocation criteria.

Allocate scholarships within a fixed budget.

Display the amount awarded to each student and the remaining budget.

Demonstrate Python concepts such as lists, dictionaries, loops, conditionals, functions, and sorting.

Algorithm:

Start the program.

Read the scholarship budget and student application details.

Check each student's academic score and annual family income.

Keep only eligible students.

Sort eligible students by academic score (descending), then family income (ascending).

Award up to ₹10,000 per student while funds remain.

Display eligible students, award amounts, and the unused budget.

Stop.

Python Program:

```
def allocate_scholarships(students, budget):
  MIN_SCORE = 70
  MAX_INCOME = 300000
  AWARD_AMOUNT = 10000

  eligible = [
    student for student in students
    if student["score"] >= MIN_SCORE
```

```
        and student["income"] <= MAX_INCOME
    ]

    eligible.sort(key=lambda student: (-student["score"], student["income"]))

    results = []
    remaining_budget = budget

    for student in eligible:
        if remaining_budget <= 0:
            break

        award = min(AWARD_AMOUNT, remaining_budget)
        results.append({
            "name": student["name"],
            "score": student["score"],
            "income": student["income"],
            "award": award
        })
        remaining_budget -= award

    return results, remaining_budget


def main():
    students = [
        {"name": "Ananya", "score": 92, "income": 180000},
        {"name": "Rahul", "score": 85, "income": 250000},
        {"name": "Meera", "score": 78, "income": 200000},
        {"name": "Arjun", "score": 68, "income": 150000},
        {"name": "Kavya", "score": 88, "income": 350000},
        {"name": "Imran", "score": 74, "income": 280000},
        {"name": "Divya", "score": 81, "income": 220000},
        {"name": "Sanjay", "score": 90, "income": 190000},
        {"name": "Pooja", "score": 73, "income": 290000},
        {"name": "Vikram", "score": 95, "income": 175000},
        {"name": "Nisha", "score": 84, "income": 240000},
        {"name": "Kiran", "score": 76, "income": 270000},
    ]

    budget = 100000
```

```
    awards, remaining = allocate_scholarships(students, budget)

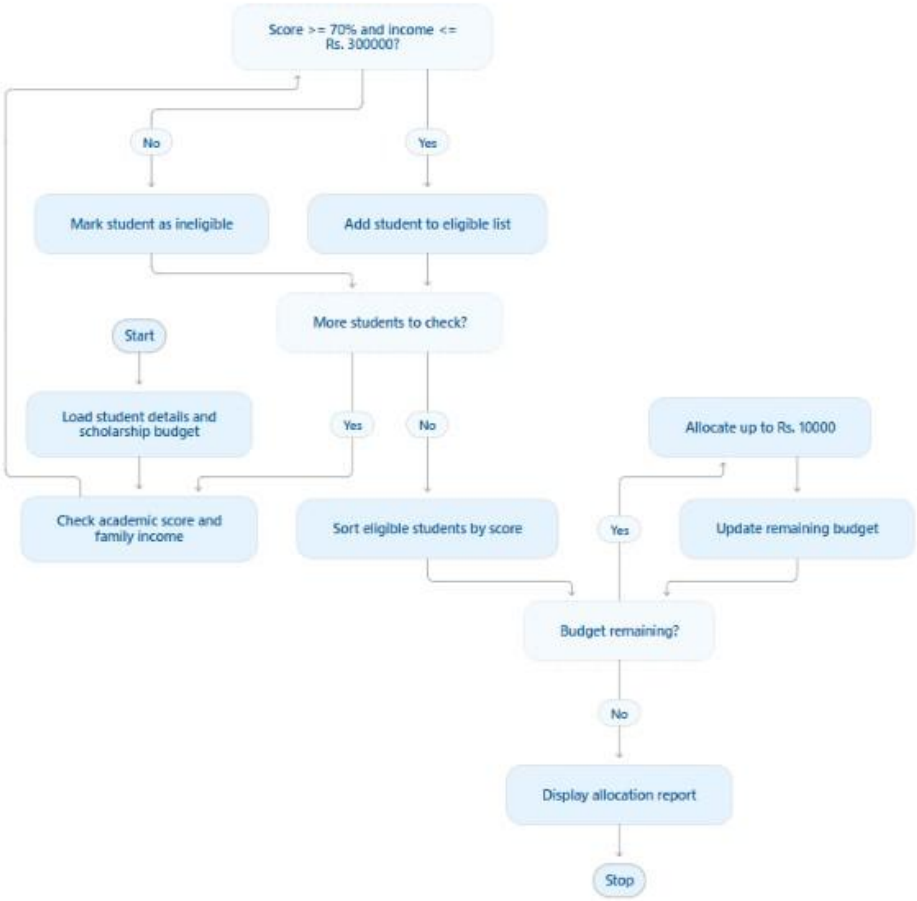
    print("\nUNIVERSITY SCHOLARSHIP ALLOCATION REPORT")
    print("-" * 65)
    print(f"{'Student':<12}{'Score':>8}{'Family Income':>18}{'Award':>15}")
    print("-" * 65)

    for student in awards:
        print(
            f"{student['name']:<12}"
            f"{student['score']:>7}%"
            f"{student['income']:>18,}"
            f"{student['award']:>15,}"
        )

    total_awarded = sum(student["award"] for student in awards)
    print("-" * 65)
    print(f"Total awarded: ₹{total_awarded:,}")
    print(f"Remaining budget: ₹{remaining:,}")
    print(f"Students awarded: {len(awards)}")


if __name__ == "__main__":
    main()
```

Flowchart:



Sample Output:

```
def allocate_scholarships(students, budget):  1 usage
    MIN_SCORE = 70
    MAX_INCOME = 300000
    AWARD_AMOUNT = 10000
    eligible = [
        student for student in students
        if student["score"] >= MIN_SCORE
        and student["income"] <= MAX_INCOME
    ]
    eligible.sort(
        key=lambda student :(_getitem_) : (
            -student["score"],
            student["income"]
        )
    )
    results = []
    remaining_budget = budget
    for student in eligible:
        if remaining_budget <= 0:
            break
        award = min(AWARD_AMOUNT, remaining_budget)
        results.append({
            "name": student["name"],
            "score": student["score"],
            "income": student["income"],
            "award": award
        })
        remaining_budget -= award
    return results, remaining_budget
def main():  1 usage
    students = [
        {"name": "Ananya", "score": 92, "income": 180000},
```

```
Run   cs-3 ×  cs-4 ×  cs-4 ×  cs-9 ×

"C:\Users\SRI MANASWINI\PycharmProjects\PythonProject1\.venv\Scripts\pyt

UNIVERSITY SCHOLARSHIP ALLOCATION
------------------------------------------------------------------
Student        Score     Family Income          Award
------------------------------------------------------------------
Vikram           95%          175,000         10,000
Ananya           92%          180,000         10,000
Sanjay           90%          190,000         10,000
Rahul            85%          250,000         10,000
Nisha            84%          240,000         10,000
Divya            81%          220,000         10,000
Meera            78%          200,000         10,000
Kiran            76%          270,000         10,000
Imran            74%          280,000         10,000
Pooja            73%          290,000         10,000
------------------------------------------------------------------
Total awarded: Rs. 100,000
Remaining budget: Rs. 0
Students awarded: 10

Process finished with exit code 0
```

Conclusion:

The University Scholarship Allocation project demonstrates how Python can support a fair, consistent, and budget-aware scholarship process. By applying clear eligibility rules and a defined priority order, the program produces an allocation report that can be adapted to a university's approved policy. Real deployments should use verified application data, protect student privacy, and be reviewed by authorized scholarship administrators.

10.University Admission Decision Engine

Problem Statement:

Universities receive many applications, and manually checking basic eligibility can take time. This project automates a simplified first-stage screening process by applying the same stated rules to each application. The result is only a sample recommendation; real admissions decisions must follow the institution's approved policy and include any required human review.

Objectives:

Evaluate student applications using consistent criteria.

Check minimum academic and entrance-exam requirements.

Classify applications as Eligible for Admission or Not Eligible.

Generate a clear decision report.

Practice Python variables, functions, conditionals, input validation, and formatted output.

Algorithm:

Start the program.

Ask the applicant to enter their name, qualifying-exam percentage, and entrance-exam score.

Validate that both scores are within the range 0–100.

Check whether the qualifying percentage is at least 60 and the entrance score is at least 50.

If both conditions are true, display “Eligible for Admission”; otherwise display “Not Eligible”.

Display the decision and the reason for the result.

Stop.

Python Program:

```
def decide_admission(name, academic_percentage, entrance_score):
  if not 0 <= academic_percentage <= 100:
    raise ValueError("Academic percentage must be between 0 and 100.")
  if not 0 <= entrance_score <= 100:
    raise ValueError("Entrance score must be between 0 and 100.")
```

```
    MIN_ACADEMIC_PERCENTAGE = 60
    MIN_ENTRANCE_SCORE = 50

    reasons = []

    if academic_percentage < MIN_ACADEMIC_PERCENTAGE:
        reasons.append("Academic percentage is below 60%.")
    if entrance_score < MIN_ENTRANCE_SCORE:
        reasons.append("Entrance score is below 50.")

    if not reasons:
        decision = "Eligible for Admission"
        explanation = "All sample eligibility criteria are satisfied."
    else:
        decision = "Not Eligible"
        explanation = " ".join(reasons)

    return {
        "name": name,
        "academic_percentage": academic_percentage,
        "entrance_score": entrance_score,
        "decision": decision,
        "explanation": explanation
    }


def main():
    print("UNIVERSITY ADMISSION DECISION ENGINE")
    print("-" * 40)

    name = input("Enter student name: ").strip()
    if not name:
        print("Error: Student name cannot be empty.")
        return

    try:
        academic_percentage = float(
            input("Enter qualifying-exam percentage (0-100): ")
        )
        entrance_score = float(
            input("Enter entrance-exam score (0-100): ")
```

```
        )

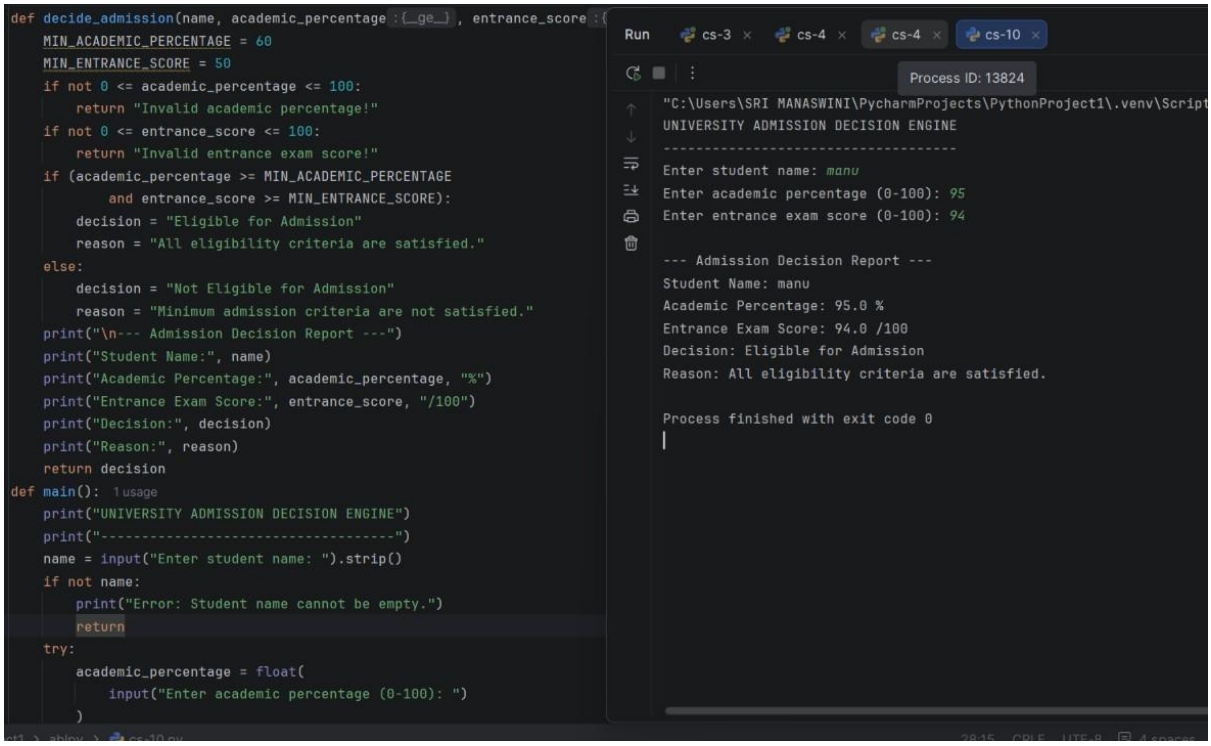
        result = decide_admission(
            name, academic_percentage, entrance_score
        )

    except ValueError as error:
        print(f"Invalid input: {error}")
        return

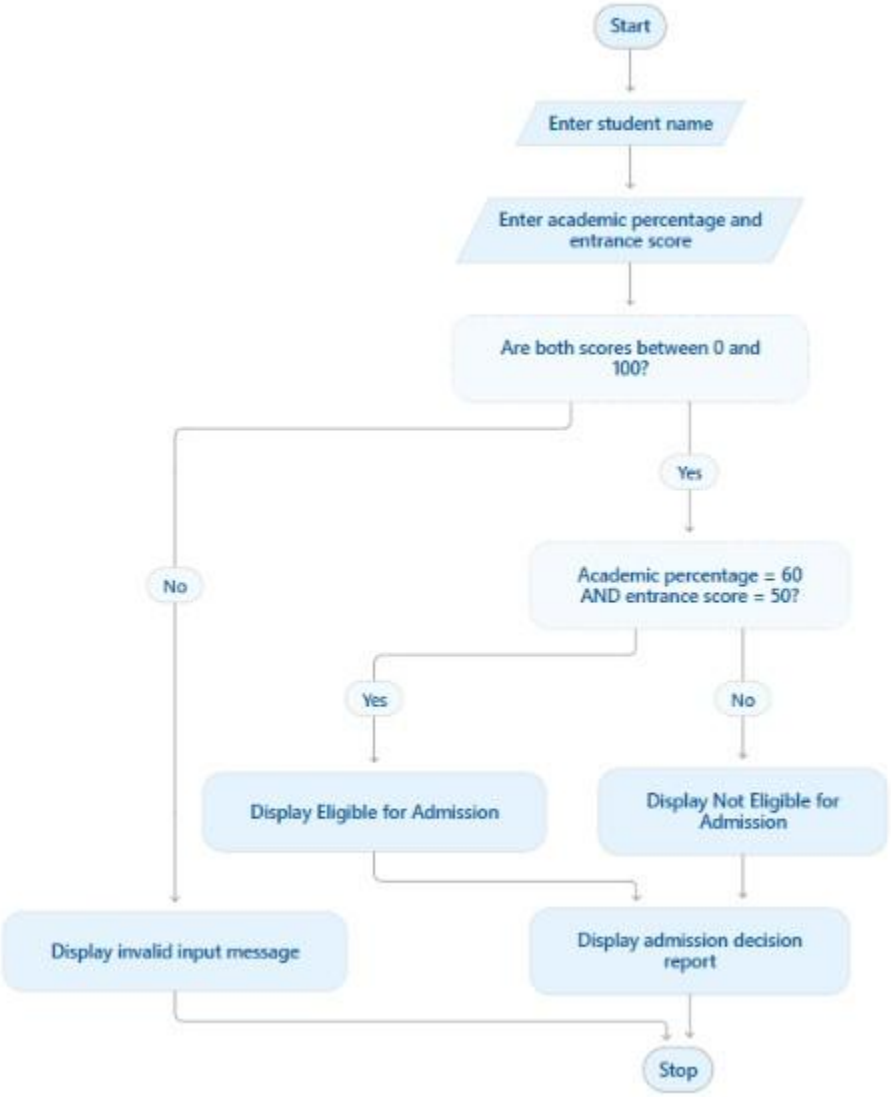
    print("\nADMISSION DECISION REPORT")
    print("-" * 40)
    print(f"Student Name: {result['name']}")
    print(f"Academic Percentage: {result['academic_percentage']:.1f}%")
    print(f"Entrance Score: {result['entrance_score']:.1f}/100")
    print(f"Decision: {result['decision']}")
    print(f"Reason: {result['explanation']}")


if __name__ == "__main__":
    main()
```

Sample Output:



Flowchart:



Conclusion:

The University Admission Decision Engine demonstrates how Python can automate a simple eligibility check with consistent rules and clear explanations. The sample thresholds are for learning only. Actual admission policies may differ by institution and program, and a real system should protect applicant data, support accessibility, and retain appropriate human oversight.